Multi-Source Data Pipelines: Ingestion, Provenance, and Honest Relationships

> Part of the Data Science / ML curriculum. The earlier documents take one dataset from raw file to a compared set of models. This document covers what changes when a product needs **several independent external sources** — measurements, reconstructions, another institution's inventory — combined into context around that one dataset: one ingestion module per source, validation before anything is published, provenance and licence as data, a harmonised layer, a careful statistical relationship between series, and scenario translation. It is written for a pipeline that runs on a schedule, unattended.

1. Why a second pipeline

The notebook pipeline in the earlier documents is **exploratory and curricular**: it reads one file, and a person reads its outputs. A context pipeline is different in kind:

	Notebook pipeline	Multi-source pipeline
Inputs	One dataset, downloaded by hand	Several external sources, each with its own release cadence, licence, and failure modes
Reader of outputs	The analyst	Other software (an API), which then serves strangers
Failure cost	The analyst notices	Nobody is watching; a silent error is published
Re-run	By hand	Monthly, from a scheduler
Therefore needs	Narrative and plots	Validation, provenance, explicit nulls, alerts, tests

Keep the two apart. Notebooks stay the place to explore and teach; stable logic that has to run unattended is extracted into plain, versioned, testable modules (Architecture Overview, §6). The pipeline writes **files** to a directory the API reads; the API never imports the pipeline (API Design, §2.3).

2. One module per source, one contract per module

Give each source its own module with the same shape:

```
fetch (or register)  →  validate  →  normalize  →  write series + provenance entry
```

- **Fetch** from a stable identifier, not a hard-coded version. If a publisher posts releases under a persistent "concept" record, resolve the latest release through it and record which one you got. If another job already downloaded a file (say, the notebook refresh), the module *registers* it — records path, checksum, row count and coverage — rather than downloading it a second time.

- **Normalize** to a boring, uniform long format: `year`, `value`, units fixed per series, explicit empty cells for "not reported."
- **Write** the series plus a **provenance entry** (§4) — never one without the other.
- Each module runs on its own from the command line (`python -m pipeline.run --source x`) so a failure in one source never blocks the others, and so it can be developed against its own fixtures.

A source you cannot publish is not deleted — it is **shelved**: excluded from the default run, writing only to an internal directory the API never reads, and kept so it can still be used to cross-check an active source locally (§4 explains the licence situation that causes this).

3. Validate before publishing

An external file can be truncated, partially updated, or silently restructured. Every module should refuse to publish something it cannot vouch for:

- **Integrity.** Verify the publisher's checksum against what you downloaded. A truncated download then fails loudly instead of producing a short series.
- **Completeness of trailing years.** The most recent year of a compiled inventory is usually *partial* — some gases or some countries have not reported. Publish a year only if it passes explicit tests, for example **emission-weighted coverage ≥ 98%** (the share of the previous year's total that is still reported) *and* a total within ±15% of the prior year. Weight by what matters rather than counting reporters: a count of reporting areas can drift by dozens while the missing ones hold 0.04% of the total. Calibrate thresholds over the whole history, so they reject the truncated year and accept every real one. Check the last few years, and trim from the **first** failing year onward (two consecutive partial years would otherwise let the second look fine relative to the first). Trim and *report*; do not extrapolate.
- **Internal consistency.** If the source publishes both parts and a total, check that the parts sum to the total within a tolerance (0.5%), every run.
- **Reconciliation across sources.** Two sources that claim the same quantity (a global total that includes international transport vs. a sum of national totals) should be compared and the *known, explained* difference stated in the output (a few percent is the shipping and aviation share), with a deviation raised if the gap exceeds a threshold.
- **Staleness.** A source file older than expected means the refresh job has stopped; a latest year more than a couple of years behind means the publisher has; a row count down more than 5% means something was dropped.

Distinguish **failures** (the source is unusable: stop, keep the last good output, exit non-zero) from **deviations from norm** (the source is usable but odd: stale, licence text changed, a year excluded, a reconciliation gap). Deviations are *warnings that become alerts* (§9); they do not abort the run.

4. Provenance and licence are data, not documentation

Write, for every series, a machine-readable provenance entry: source URLs, retrieval time, the release stamp, the raw file's SHA-256, coverage, units, scope, methodology, caveats, **licence**, and **required citation text**. Downstream, this is what lets every API response carry its attribution automatically (API Design, §3.5).

Licence handling is where pipelines quietly go wrong, so make it explicit:

- **Read the licence of the exact files you transform.** A compiled dataset can license its *own* terms permissively while embedding a component (a licensed energy-statistics input, say) under a no-derivatives clause. If your pipeline transforms and republishes that component's series, the no-derivatives term is violated even though the aggregator's terms look fine. The remedy used here: replace the source with an equivalent whose licence allows the use (non-commercial, share-alike, with the notice and attribution carried), and **shelve** the original.
- **Record the restriction in the output**, so a downstream user cannot miss it: "non-commercial use only; share-alike; cite as follows."
- **When a publisher states no licence at all,** record that verbatim ("No formal licence is stated; use is conditional on citing the original source") along with the required citation format, and make every output that uses the series carry the citation.
- **Pin the vintage caveat to a flag, not to memory.** A known discrepancy between releases ("a possible ~0.1 °C difference has not been reconciled") appears in outputs until a config flag says it has been reconciled — set by a person, with a date, when they have checked.

5. The harmonised layer: one key, one unit, explicit nulls

Combining sources means a **harmonised layer**: every indicator in one consistent shape, precomputed so nothing downstream re-derives it.

- **One key.** An integer calendar year, unique per indicator (and per country for country-scoped ones). One **unit** and one **scope** (global or country) per indicator.
- **Explicit nulls.** A missing value is an empty cell. It is never interpolated, never zero. A zero a source itself reports stays a zero (an early-history gas really can be 0.0).
- **A catalog.** `indicator_catalog` lists every indicator: id, name, unit, *kind*, its own coverage, a link to its source's provenance, and — for derived indicators — the base, the formula and the baselines that were refused and why.
- **Kind decides what you may derive.** This is the part that prevents nonsense:

Kind	Allowed derivations	Never
Level (annual totals)	Year-on-year %; a **trailing** 5-year mean (null until 5 consecutive observations; trailing so it never uses a later year); an index to each allowed baseline year, only where the baseline value exists and is > 0	Indexing to a baseline of zero
Cumulative	Nothing beyond itself	Indexing or averaging
Anomaly (relative to a reference period)	Trailing mean; both native references and the interval	Indexing ("= 100" is meaningless when the value is near zero or negative) and year-on-year % (undefined)
Uncertainty	Describes a series	Being treated as a series

- **Scope-limited.** Keep the country layer deliberately small (total, per-gas, cumulative per area). Every added indicator is another thing to validate, document and keep correct.

6. Pairing series for analysis

Before any relationship is fitted, **align** the two series with a function whose rules are conservative and visible:

```
frame, meta = align_pair(harmonized, "cumulative_emissions", "temperature_anomaly")
```

- Keep only years where **both** series have a value (no interpolation).
- Return `meta` with both indicators' catalog entries, the requested, common and used ranges, **every omitted year with its reason** (`a missing` / `b missing` / `both missing`), `interpolated: false`, and a standing note that co-movement is interpretive context, not proof of causation.
- **Refuse** with a clear error: pairing an indicator with itself; a global series with a country series (a country's emissions are never paired with the *global* temperature line — country pairs need a named geography shared by both); an uncertainty series; an empty range; and fewer than a minimum number of shared years (20). A correlation over a handful of points looks authoritative and means nothing.

7. Fitting a relationship honestly

The goal is to describe how much a quantity changes per unit of another (here: warming per unit of cumulative emissions) in a way that survives scrutiny. Several decisions make the difference between a defensible number and a decorative one:

1. **Choose the regressor for a reason.** Warming tracks the *cumulative* total of the gas, not the annual flow, so regress the anomaly on cumulative emissions. The slope does not depend on where the cumulative starts (a constant added to *x* moves only the intercept), so cumulative series that begin in different years are comparable.
2. **Don't trust plain OLS standard errors on trending series.** Residuals are autocorrelated (lag-1 ≈ 0.6, Durbin–Watson ≈ 0.8 here), so ordinary errors are too narrow. Use **heteroskedasticity-and-autocorrelation-consistent (Newey–West / HAC)** errors for the confidence interval, with a stated bandwidth rule — and publish how the interval moves under other bandwidths.
3. **Add a resampling check that respects time.** A **moving-block bootstrap of the residuals** (predictor held fixed, blocks of consecutive years, a *fixed recorded seed* so it is identical run to run) gives a second interval. A pairs bootstrap is wrong here: on a trending *x* it resamples samples with almost no spread in *x*. Build blocks only from calendar-consecutive years — if a gap exists, row-based blocks would span it (and Newey–West lags count rows, so a gap is itself a deviation). Validate the method on simulated data with a known slope: it should cover the truth close to the nominal rate (92% against 95% nominal is fine; 60% is not).
4. **Hold out decades.** Fit on the years before 1980/1990/2000/2010, predict the rest, and report RMSE and MAE next to the RMSE of just predicting the training mean. An in-sample R^2 of 0.9 on two trending series says almost nothing.

5. **Publish the sensitivities, don't hide them.** The slope by estimation window (and say if it moves away from the reference value as the window starts later), the interval under several HAC bandwidths and block lengths, and the effect of an uncertain input (scale a modelled component by 0.7/1.0/1.3 and report each slope).
6. **Report awkward findings the way you'd want them reported.** If the "more complete" variant has *worse* out-of-sample error than the simpler one, say so in generated text built from the numbers — and state that the cause is not established. Do not tune a choice (a weight, a window) to minimise the holdout error; publish the scan to show the shape, not to pick a value.
7. **Don't borrow a reference's name for a different quantity.** A published reference range (a climate-science consensus on warming per unit of CO_2) applies to the CO_2-only quantity. A related regression on all gases, a short window, and CO_2-equivalent weights is a different thing: no comparison to that range, never called by its name, and its caveats say why. A test that scans the output files for the reserved word enforces it.
8. **No pass/fail label.** Generate a plain-language summary of the numbers instead.
9. **Descriptions must be generated from the numbers.** Every sentence that quotes a figure ("a larger error than the fossil-only variant (0.21 vs 0.10 °C)") is built from the same values the chart uses, so the UI, the agent and the docs cannot disagree.

8. Translating scenarios into implied outcomes

A scenario projection (three emissions pathways) can be translated into implied outcomes using the fitted slope. Treat that as an **illustrative translation, not a model**, and make every assumption an explicit, published field:

- **Cover the part of the world you modelled and hold the rest fixed**: global = covered ÷ (1 − s), where *s* is the rest-of-world's last-observed share. State the share.
- **Match the slope's definition.** If the slope is per unit of *total* emissions including a component the scenarios don't model (land use), hold that component at a stated value (its trailing 5-year mean) and say so. Publish a second line using the narrower slope with no such assumption so the effect of the definition is visible.
- **Warming is incremental**: ΔT = slope × cumulative emissions since the last observed year, with a band from the slope's confidence interval — *slope uncertainty only*, and the file says so — added to an **anchor** that is a trailing mean of recent observations, not a single noisy year and not the regression line.
- **Check the seam.** The first projected year must be compared with the last observed value, in aggregate (**±2%**) and per entity (**±5%**), and the step reported. An aggregate breach is a deviation; per-entity breaches are a deviation only when systematic (here, more than 10% of entities) and otherwise a note: one anomalous country should not page you monthly. A scenario that begins 3.7% above the last observation fails the sanity check on day one.
- **Guard against stale scenarios.** If the scenario file is produced by a different, less frequent job, check that its first year is the last observed year + 1, that the entity × year grid is complete, and that every entity has a value at the last year. Otherwise publish explicit nulls with the reason ("scenarios are stale; rerun the notebook"), never a stale translation.
- **Say why the answer looks surprising.** If scenarios whose emissions differ 2.2× produce outcomes only 0.1 °C apart, generate a reading note from the figures (most of the final level is warming already observed; the scenarios only change what is yet to come) rather than letting a reader think the model is broken.

The baseline lesson (two fits, not one). A forecast fitted only through a cutoff (so the following years can be held out for evaluation — Time-Series Forecasting, §9) is the right teaching artifact and the wrong production baseline: reused as a scenario starting point it ignores everything after the cutoff. Keep both: the evaluation fit unchanged, and a second fit on all observed years for production use, checked at the seam as above and backtested at two earlier cutoffs. Flag single-year data anomalies (a war-time spike at the start of a training window can dominate one country's smoothing parameters) rather than altering them, and say what the backtest does *not* show (skill over a 19-year horizon).

9. Operating it: scheduled refresh and alerts

A pipeline that runs monthly without a person watching needs operational habits built in:

- **Every output is always rewritten.** When an input is missing, malformed, stale or refused, the stage writes the **same schema with explicit nulls** plus an `unavailable_reason` and a deviation — never leaves the previous file in place. An old regression sitting under a new timestamp is worse than no regression. A test should compare the key set of an unavailable output with an available one so the shapes cannot diverge, and parse every output with a strict JSON parser (`NaN` and `Infinity` are not JSON).
- **Failure domains.** A stage that merges independent sources (harmonisation) does not gate on any one of them — each source's age is in its provenance. A stage that *reads* another stage's output declares the dependency (`DEPENDS_ON`) and is **skipped, recorded as a failure**, when that stage failed in the same run, so a fresh-looking result is never built from the previous run's stale inputs. Run on its own (`--source x`) a stage is never gated.
- **Back up, validate, restore.** The job backs up the prior download, validates the new file against it (hard-fail thresholds restore the backup with its modification time; soft-flag thresholds publish and alert), then runs the pipeline.
- **Restart what caches.** After a validated refresh, restart the API so its in-process caches see the new files (API Deployment, §4.1); a failed restart is reported, never fatal.
- **One notification per run**, whose priority is the highest of the notebook outcome and the pipeline summary: urgent if a source failed outright, high if there are deviations, default if clean. The pipeline writes `last_run.json` plus the title/message/priority the job forwards.
- **Canary your environment.** A library bug can silently corrupt results with no error — for example, a dense pivot on more than ~32,000 fully populated rows returning duplicated labels under one numpy/Python combination. Pin the fixed version, avoid the construct where you can (build frames from arrays), and keep a small canary check that reports an `environment` deviation if the bug is ever present again.
- **Nothing deploys by merging.** Versioned job scripts and scheduler definitions live in the repo, but installing them on the host is an explicit, separate step.

10. Testing a pipeline

- **Offline.** No network in tests: each source module takes its fetch function as a parameter or reads from a stub, and fixtures are tiny hand-built files.
- **Per-source validation tests.** A truncated file, a bad checksum, a trailing partial year, a gap in the middle, a duplicated year — each must be refused or trimmed as designed.

- **Know-the-answer fixtures** for the statistics: simulate data with a known slope and assert the point estimate and coverage; assert holdout RMSE against a hand calculation on a 3-point example.
- **Schema-completeness tests** for the unavailable path (§9), **scan tests** for reserved wording (the word reserved for another quantity must not appear), and **strict-JSON tests**.
- **Consumers test against the real producer** — see API Testing, §4.3.

11. Core libraries summary

Library	Used for
pandas / **NumPy**	tabular normalization; array-based construction where dense reshapes are risky
statsmodels	OLS with HAC (Newey–West) covariance; import from the specific submodules the pinned scipy supports
requests / hashlib	fetching with checksum verification and recording SHA-256
JSON + CSV	the hand-off format: a small metadata JSON next to a long-format CSV per result

See also

- Data Science / ML index
- EDA & Data Engineering, §9 — the bronze/silver/gold vocabulary; the harmonised layer here is silver-to-gold for several sources
- Time-Series Forecasting — the forecasting model whose second, all-data fit supplies the scenario baseline (§8)
- Python API Backend, Design §2.3 — how the API consumes these files and fails closed